

Tema 6 — Elementos de programación funcional

Laboratorio Guiado

Objetivos	Practicar el uso de elementos de programación funcional en Python: funciones como objetos, funciones de orden superior, map(), filter(), lambda, reduce(), sorted(key=...), any(), all(), expresiones generadoras y pipelines sencillos de datos.
Material necesario	Terminal, Python 3.13, venv del Tema06, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en ~/Curso_Python/Tema06.

Laboratorio 1. Funciones como objetos

1. Actualizar el repositorio y abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
git pull origin main
cd ~/Curso_Python/Tema06
source .venv/bin/activate
python --version
code .
```

2. Abrir el fichero 01_funciones_como_objetos.py. Revisar una función pequeña que recibe un dato y devuelve un nuevo resultado sin modificar el valor original.

```
def normalizar_servicio(nombre):
    """Devuelve el nombre de un servicio sin espacios y en minúsculas."""
    return nombre.strip().lower()

servicio_original = "  SSH  "
servicio_normalizado = normalizar_servicio(servicio_original)

print("Original:", servicio_original)
print("Normalizado:", servicio_normalizado)
```

3. Comprobar que una función se puede asignar a una variable. No se escriben paréntesis en la asignación porque no queremos ejecutarla todavía.

```
operacion = normalizar_servicio

print(operacion("  NGINX  "))
print(type(operacion))
```

4. Revisar una función que recibe otra función como argumento. `aplicar_a_servicios()` separa el recorrido de la operación concreta que se aplica.

```
def aplicar_a_servicios(servicios, funcion):
    """Aplica una función a cada servicio recibido."""
    resultado = []

    for servicio in servicios:
        resultado.append(funcion(servicio))

    return resultado

servicios = [" SSH ", " API ", " NGINX "]
limpios = aplicar_a_servicios(servicios, normalizar_servicio)

print("Entrada:", servicios)
print("Salida:", limpios)
```

5. Analizar una función anidada. La función interna `media()` solo existe dentro de `resumen_cpu()`, por lo que no contamina el ámbito global.

```
def resumen_cpu(valores):
    """Devuelve la media y el máximo de una lista de mediciones de CPU."""
    def media(datos):
        return sum(datos) / len(datos)

    promedio = media(valores)
    maximo = max(valores)
    return promedio, maximo

promedio_cpu, maximo_cpu = resumen_cpu([20, 35, 80])
print("Promedio CPU:", round(promedio_cpu, 2))
print("Máximo CPU:", maximo_cpu)
```

6. Revisar una función que devuelve otra función. La función interna recuerda los valores mínimo y máximo recibidos por la función externa.

```
def crear_validador_puerto(minimo, maximo):
    """Devuelve una función que valida si un puerto está dentro de un rango."""
    def validar(puerto):
        return minimo <= puerto <= maximo

    return validar

puerto_valido = crear_validador_puerto(1, 65535)

print("Puerto 443 válido:", puerto_valido(443))
print("Puerto 70000 válido:", puerto_valido(70000))
```

7. Ejecutar el script completo y comprobar que las funciones se tratan como objetos reutilizables.

```
python 01_funciones_como_objetos.py
```

Resultado esperado: el alumno debe identificar que una función puede guardarse en una variable, pasarse a otra función y devolverse como resultado.

Laboratorio 2. Transformaciones y filtros con map() y filter()

1. Abrir el fichero 02_map_filter.py. Revisar map() como mecanismo para aplicar una función a cada elemento de una colección.

```
def normalizar_servicio(servicio):  
    """Normaliza un nombre de servicio eliminando espacios y usando minúsculas."""  
    return servicio.strip().lower()  
  
servicios = [" SSH ", " NGINX ", " Api "]  
  
normalizados_iterable = map(normalizar_servicio, servicios)  
print("Objeto devuelto por map:", normalizados_iterable)  
  
normalizados = list(normalizados_iterable)  
  
print("Normalizados:", normalizados)  
print("Original:", servicios)
```

2. Comprobar la evaluación perezosa. map() devuelve un iterable que produce valores cuando se recorre; si se consume, no se puede volver a leer completo.

```
servicios = ["ssh", "nginx", "api"]  
resultado = map(str.upper, servicios)  
  
print("Primera conversión:", list(resultado))  
print("Segunda conversión:", list(resultado))  
print("La segunda lista sale vacía porque el iterable ya se consumió.")
```

3. Usar map() con dos colecciones. La función debe aceptar tantos argumentos como colecciones se le pasen a map().

```
def crear_endpoint(servicio, puerto):  
    """Construye una cadena servicio:puerto."""  
    return f"{servicio}:{puerto}"  
  
servicios = ["ssh", "nginx", "postgresql"]  
puertos = [22, 443, 5432]  
  
endpoints = list(map(crear_endpoint, servicios, puertos))  
print("Endpoints:", endpoints)
```

4. Revisar filter(). La función predicado devuelve True o False, y filter() conserva solo los elementos que cumplen la condición.

```
def esta_activo(servicio):  
    """Devuelve True si el servicio está activo."""  
    return servicio["activo"]  
  
servicios = [  
    {"nombre": "ssh", "activo": True},  
    {"nombre": "api", "activo": False},  
    {"nombre": "http", "activo": False},  
    {"nombre": "ntp", "activo": True},  
]  
  
activos = list(filter(esta_activo, servicios))
```

```
print("Servicios activos:", activos)
```

5. Filtrar un diccionario usando `items()`. Cada item recibido es una tupla con clave y valor, y después se reconstruye el diccionario con `dict()`.

```
puertos = {
    "ssh": 22,
    "nginx": 443,
    "api": 8080,
    "rdp": 3389,
}

def es_puerto_estandar(item):
    """Devuelve True si el puerto del item es habitual o estándar."""
    servicio, puerto = item
    return puerto in (22, 80, 443)

puertos_estandar = dict(filter(es_puerto_estandar, puertos.items()))
print("Puertos estándar:", puertos_estandar)
```

6. Comparar una comprensión de listas con `map()`. Cuando la operación es simple, la comprensión suele ser más legible.

```
servicios = [" SSH ", " Api ", " NGINX "]

limpios_1 = [servicio.strip().lower() for servicio in servicios]
limpios_2 = list(map(normalizar_servicio, servicios))

print("Comprensión:", limpios_1)
print("map():", limpios_2)
```

7. Ejecutar el script completo y revisar la salida.

```
python 02_map_filter.py
```

Resultado esperado: el alumno debe distinguir transformación con `map()`, filtrado con `filter()`, materialización con `list()` y consumo de iterables.

Laboratorio 3. `lambda`, `sorted()`, `any()`, `all()` y expresiones generadoras

1. Abrir el fichero `03_lambda_sorted_any_all_generadores.py`. Revisar una `lambda` breve aplicada con `map()` y `filter()`.

```
servicios = ["ssh", "nginx", "postgresql", "api"]

longitudes = list(map(lambda servicio: len(servicio), servicios))
print("Longitudes:", longitudes)

sin_api = list(filter(lambda servicio: servicio != "api", servicios))
print("Servicios excepto api:", sin_api)
```

2. Comparar lambda con def. Si la lógica necesita varias instrucciones o una explicación clara, conviene crear una función con nombre.

```
def describir_puerto(puerto):  
    """Devuelve una descripción breve de un puerto."""  
    if puerto in (80, 443):  
        return f"{puerto}: web"  
    return f"{puerto}: otro"  
  
puertos = [22, 80, 443, 8080]  
descripciones = list(map(describir_puerto, puertos))  
  
print(descripciones)
```

3. Ordenar una lista de diccionarios con sorted(key=...). La función key calcula el criterio de ordenación para cada elemento.

```
servicios = [  
    {"nombre": "ssh", "puerto": 22},  
    {"nombre": "api", "puerto": 8080},  
    {"nombre": "nginx", "puerto": 443},  
    {"nombre": "rdp", "puerto": 3389},  
]  
  
ordenados_por_puerto = sorted(servicios, key=lambda servicio: servicio["puerto"])  
  
for servicio in ordenados_por_puerto:  
    print(servicio)
```

4. Usar any() y all() para validar una colección sin escribir bucles explícitos.

```
puertos = [22, 443, 8080]  
  
hay_puerto_alto = any(puerto > 1024 for puerto in puertos)  
todos_validos = all(1 <= puerto <= 65535 for puerto in puertos)  
  
print("Hay puerto alto:", hay_puerto_alto)  
print("Todos los puertos son válidos:", todos_validos)
```

5. Crear una expresión generadora. A diferencia de una lista por comprensión, el generador produce valores bajo demanda.

```
logs = ["OK ssh", "ERROR api", "ERROR nginx"]  
  
errores = (  
    linea for linea in logs  
    if linea.startswith("ERROR")  
)  
  
print("Objeto generador:", errores)  
  
for error in errores:  
    print("Error detectado:", error)
```

6. Comprobar que un generador también se consume después de recorrerlo.

```
errores = (  
    linea for linea in logs  
    if linea.startswith("ERROR")  
)  
  
print("Primera lectura:", list(errores))  
print("Segunda lectura:", list(errores))
```

7. Ejecutar el script completo.

```
python 03_lambda_sorted_any_all_generadores.py
```

Resultado esperado: el alumno debe usar lambda solo en expresiones breves y reconocer sorted(key=...), any(), all() y generadores.

Laboratorio 4. reduce() y acumulaciones

1. Abrir el fichero 04_reduce_y_acumulaciones.py. Importar reduce() desde functools y revisar una acumulación sencilla.

```
from functools import reduce  
  
medidas = [12, 18, 7, 20]  
  
def acumular(total, valor):  
    """Suma el valor actual al acumulador."""  
    return total + valor  
  
suma_reduce = reduce(acumular, medidas)  
suma_sum = sum(medidas)  
  
print("Suma con reduce:", suma_reduce)  
print("Suma con sum:", suma_sum)
```

2. Usar reduce() para concatenar texto. En cada iteración se recibe un acumulador parcial y el siguiente elemento.

```
nombres = ["ssh", "rdp", "www", "ntp"]  
  
def concatenar(acumulador, nombre):  
    """Concatena nombres en mayúsculas separados por dos puntos."""  
    return acumulador.upper() + " : " + nombre.upper()  
  
cadena = reduce(concatenar, nombres)  
print(cadena)
```

3. Revisar `reduce()` con valor inicial. El valor inicial define el primer acumulador y evita problemas con colecciones vacías.

```
puertos = [8080, 22, 8443, 443, 1024]

maximo = reduce(
    lambda acumulador, puerto: puerto if puerto > acumulador else acumulador,
    puertos,
    0,
)

print("Puerto máximo:", maximo)
```

4. Comprobar una colección vacía con valor inicial y comparar con una alternativa más clara usando `max()`.

```
sin_puertos = []

maximo_vacio = reduce(
    lambda acumulador, puerto: puerto if puerto > acumulador else acumulador,
    sin_puertos,
    0,
)

print("Puerto máximo en lista vacía:", maximo_vacio)
print("Puerto máximo con max():", max(puertos))
```

5. Usar `reduce()` para construir un diccionario contador. Este caso modifica el acumulador de forma explícita y controlada.

```
estados = ["OK", "ERROR", "OK", "REVISAR", "ERROR"]

def contar_estados(conteo, estado):
    """Actualiza un diccionario contador."""
    if estado not in conteo:
        conteo[estado] = 0

    conteo[estado] += 1
    return conteo

conteo = reduce(contar_estados, estados, {})
print("Conteo de estados:", conteo)
```

6. Ejecutar el script completo.

```
python 04_reduce_y_acumulaciones.py
```

Resultado esperado: el alumno debe comprender `reduce()` como acumulación y valorar cuándo `sum()`, `max()`, `min()` o un bucle son más claros.

Laboratorio 5. Pipeline funcional integrado

1. Abrir el fichero 05_pipeline_funcional_integrado.py. Revisar los datos de entrada simulando líneas de inventario.

```
lineas = [  
    " SSH ; 22 ; activo ",  
    " nginx ; 443 ; activo ",  
    " api ; 8080 ; detenido ",  
    " backup ; 70000 ; activo ",  
    " rdp ; 3389 ; activo ",  
    " ntp ; 123 ; activo ",  
]  
  
for linea in lineas:  
    print(repr(linea))
```

2. Revisar las funciones auxiliares. Cada función resuelve un paso pequeño del flujo: limpiar, dividir, construir, validar o describir.

```
def limpiar_linea(linea):  
    """Elimina espacios sobrantes de la línea completa."""  
    return linea.strip()  
  
def dividir_linea(linea):  
    """Divide una línea en campos."""  
    nombre, puerto, estado = linea.split(";")  
    return nombre.strip().lower(), puerto.strip(), estado.strip().lower()  
  
def construir_servicio(campos):  
    """Convierte una tupla de campos en un diccionario de servicio."""  
    nombre, puerto, estado = campos  
    return {  
        "nombre": nombre,  
        "puerto": int(puerto),  
        "activo": estado == "activo",  
    }
```

3. Completar las funciones de validación, filtrado y presentación.

```
def puerto_valido(servicio):  
    """Devuelve True si el puerto del servicio está en el rango válido."""  
    return 1 <= servicio["puerto"] <= 65535  
  
def servicio_activo(servicio):  
    """Devuelve True si el servicio está marcado como activo."""  
    return servicio["activo"]  
  
def describir_servicio(servicio):  
    """Devuelve una cadena legible para el reporte."""  
    return f'{servicio["nombre"]}:{servicio["puerto"]}'
```


4. Aplicar map() para limpiar y convertir los datos. Cada paso transforma la salida del paso anterior.

```
limpias = list(map(limpiar_linea, lineas))
campos = list(map(dividir_linea, limpias))
servicios = list(map(construir_servicio, campos))

print("Líneas limpias:", limpias)
print("Campos:", campos)
print("Servicios:", servicios)
```

5. Aplicar filter() para quedarse con servicios activos y con puertos válidos.

```
activos = list(filter(servicio_activo, servicios))
activos_validos = list(filter(puerto_valido, activos))

print("Servicios activos:", activos)
print("Servicios activos con puerto válido:", activos_validos)
```

6. Ordenar los servicios válidos y construir un reporte legible con map().

```
ordenados = sorted(activos_validos, key=lambda servicio: servicio["puerto"])
reporte = list(map(describir_servicio, ordenados))

print("Reporte:")
for linea in reporte:
    print("-", linea)
```

7. Usar any(), all() y reduce() para obtener validaciones y un resumen de estados.

```
from functools import reduce

hay_puertos_altos = any(servicio["puerto"] > 1024 for servicio in activos_validos)
todos_validos = all(puerto_valido(servicio) for servicio in activos_validos)

print("Hay puertos altos:", hay_puertos_altos)
print("Todos los activos filtrados tienen puertos válidos:", todos_validos)

def contar_por_estado(conteo, servicio):
    """Acumula cuántos servicios hay por estado lógico."""
    estado = "activo" if servicio["activo"] else "detenido"
    if estado not in conteo:
        conteo[estado] = 0
    conteo[estado] += 1
    return conteo

resumen_estados = reduce(contar_por_estado, servicios, {})
print("Resumen de estados:", resumen_estados)
```

8. Ejecutar el script completo.

```
python 05_pipeline_funcional_integrado.py
```

Resultado esperado: el alumno debe ver cómo encadenar transformaciones, filtros, ordenación, validación y acumulación sin crear un bloque monolítico difícil de mantener.

Ejercicios propuestos

1. Crear una función `normalizar_usuario()` y usar `map()` para limpiar una lista de usuarios con espacios y mayúsculas mezcladas.
2. Dada una lista de puertos, usar `filter()` para obtener solo los puertos válidos entre 1 y 65535.
3. Ordenar una lista de diccionarios de servidores por `hostname` y después por número de servicios activos.
4. Usar `any()` para detectar si existe algún servicio detenido y `all()` para comprobar si todos los puertos son válidos.
5. Crear un pequeño pipeline que reciba líneas de log, filtre las que empiezan por `ERROR` y genere una lista con los nombres de los servicios afectados.
6. Rehacer una operación implementada con `map()` o `filter()` usando una comprensión de listas y comparar cuál resulta más legible.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Curso_Python) en el prompt	El entorno virtual no está activado.	Ejecutar <code>source .venv/bin/activate</code> desde <code>~/Curso_Python/Tema06</code> .
El script no se encuentra	La terminal no está situada en el directorio del tema.	Ejecutar <code>cd ~/Curso_Python/Tema06</code> y comprobar <code>ls -la</code> .
<code>map()</code> o <code>filter()</code> parecen no mostrar resultados	Se está mostrando el objeto iterable, no sus valores materializados.	Convertir explícitamente a <code>list()</code> o recorrer el iterable con un bucle <code>for</code> .
Una segunda lectura devuelve vacío	<code>map()</code> , <code>filter()</code> o un generador ya fueron consumidos en una lectura anterior.	Crear de nuevo el iterable o materializar el resultado si se va a reutilizar.
Una lambda resulta difícil de leer	La expresión contiene demasiada lógica o necesita explicación adicional.	Sustituirla por una función definida con <code>def</code> y un nombre descriptivo.
<code>reduce()</code> genera error con una colección vacía	No se ha indicado un valor inicial para el acumulador.	Proporcionar un valor inicial o usar <code>sum()</code> , <code>max()</code> , <code>min()</code> o un bucle cuando sea más claro.
<code>sorted(key=...)</code> no ordena como se espera	La función <code>key</code> no devuelve el campo adecuado para ordenar.	Comprobar la función <code>key</code> y probarla sobre un elemento individual.
<code>KeyError</code> o <code>ValueError</code> al procesar datos	Falta una clave en un diccionario o un texto no puede convertirse con <code>int()</code> .	Revisar las claves con <code>get()</code> y comprobar el contenido antes de convertir tipos.